AWS VPC PEERING PROJECT

Private Connectivity Between Two Amazon Virtual Private Clouds

Project information	Details
Project	EWB Courses – AWS VPC Peering Student Project
Prepared by	Kotimarpina
Platform / Region	Amazon Web Services – Asia Pacific (Mumbai), ap-south-1
Account arrangement	Same AWS account and same Region
Report date	31 October 2026
Implementation status	VPCs, EC2 instances, peering and shown VPC A routes configured; end-to-end tests need final evidence

Report purpose

This report records the network design, AWS resources, configuration steps, routing, security controls, troubleshooting, test status and cleanup procedure for the VPC Peering lab. It is written in straightforward language and uses the supplied AWS console and terminal screenshots as evidence. It clearly separates what the screenshots prove from what still needs to be tested.

Important evidence note

The peering connection is confirmed Active and the supplied route-table screenshots show Active peering routes in VPC A. The supplied screenshots do not show successful App-A/App-B ping tests or a successful SSH login to App-B. Those results are therefore marked pending and must not be represented as completed.

1. Project Overview

The purpose of this lab is to connect two isolated AWS virtual networks using VPC Peering. VPC A contains a public Bastion Host and a private application server named App-A. VPC B contains a private application server named App-B. The application instances are designed to communicate using private IPv4 addresses rather than public IP addresses.

A VPC Peering connection creates a private network path between the two VPCs. It does not automatically configure routing or security. Route tables must direct traffic to the peering connection, and security groups must permit the required traffic. Both sides need a route so that request and response packets can travel.

2. Project Objectives

• Create two VPCs with non-overlapping IPv4 CIDR blocks.
• Create the public and private subnets specified in the workbook.
• Deploy a Bastion Host in the public subnet and two private EC2 instances.
• Attach an Internet Gateway to VPC A and configure its public route table.
• Establish and accept a VPC Peering connection.
• Configure reciprocal routes for traffic between VPC A and VPC B.
• Restrict SSH and ICMP traffic using security groups.
• Test private connectivity, capture evidence and clean up lab resources.

3. Architecture and Network Address Plan

Component	**VPC A – ewb-vpc-a**	**VPC B – ewb-vpc-b**
VPC CIDR	10.10.0.0/16	10.20.0.0/16
Public subnet	ewb-a-public 10.10.1.0/24	Not used
Private subnet	ewb-a-private 10.10.2.0/24	ewb-b-private 10.20.1.0/24
EC2 instance	Bastion: 10.10.1.10 App-A: 10.10.2.10	App-B: 10.20.1.10
Public IPv4	Bastion: 13.232.237.127	No public IPv4 shown for App-B
Private application server	App-A has no public IPv4 shown	App-B has no public IPv4 shown

The VPC CIDR ranges do not overlap. This is required for VPC Peering. The Bastion is placed in the public subnet so an administrator can reach it from outside the VPC. App-A and App-B remain in private subnets.

4. Architecture Diagram

The diagram below summarizes the intended network layout, IP addressing, route tables and traffic paths. It is an architecture illustration; the actual connectivity test status is described in Section 10.

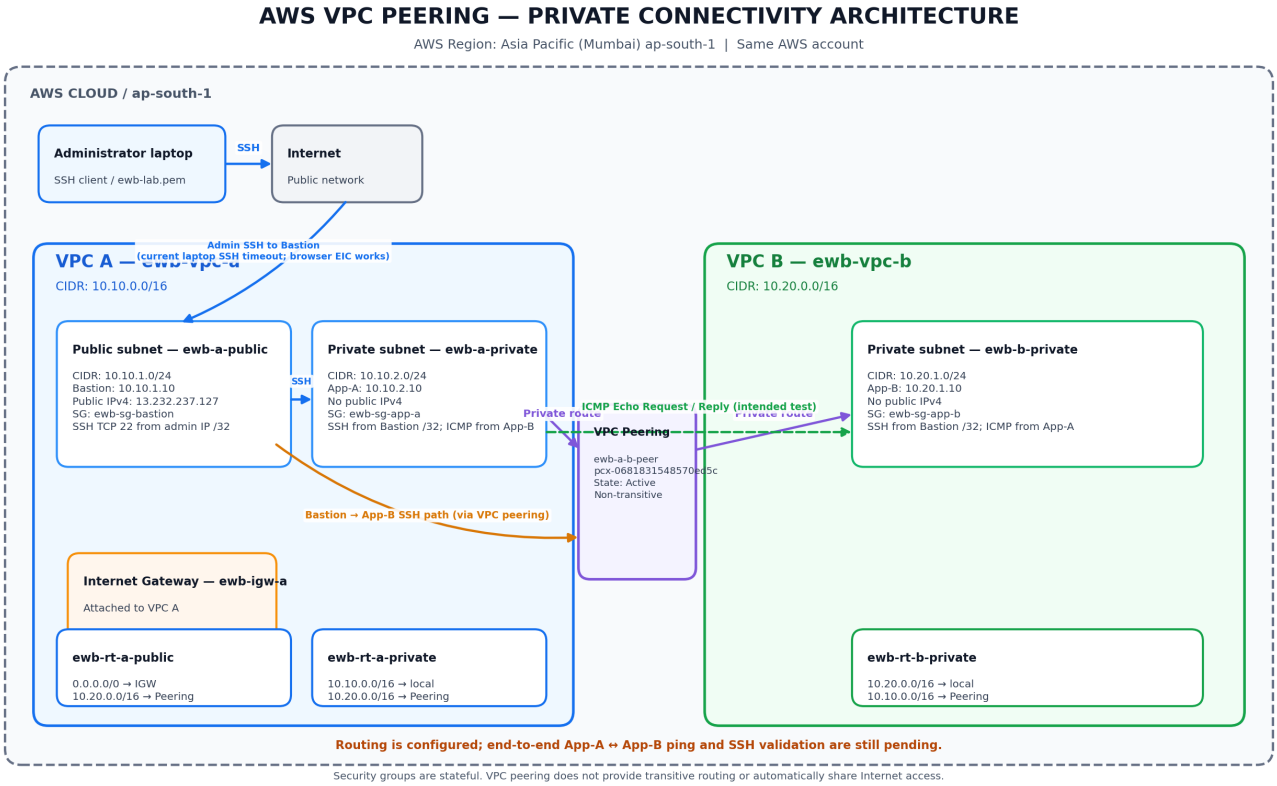


Figure 1. AWS VPC Peering architecture based on the lab configuration.

5. AWS Resource Inventory

Resource	Name / identifier / value
VPC A	ewb-vpc-a | vpc-0eac18c3eebba4b10 | 10.10.0.0/16
VPC B	ewb-vpc-b | vpc-0000dab2d7cbe0266 | 10.20.0.0/16
Peering connection	ewb-a-b-peer | pcx-0681831548570ed5c | Active
Internet Gateway	ewb-igw-a | igw-0df012962563faa97
Public route table	ewb-rt-a-public | rtb-0d321c2c0a3b5a77e
Private route table A	ewb-rt-a-private | rtb-02dc8b9be7720c442
Private route table B	ewb-rt-b-private | rtb-0e7709a0b5a6ad5aa
Bastion	ewb-bastion | i-02ea14f0727b5b0aa | t3.micro
App-A	ewb-app-a | i-071309621bfd7115d | t3.micro
App-B	ewb-app-b | i-09ea4e03b02e4532f | t3.micro
App-A security group	ewb-sg-app-a | sg-0077524c37733338a
Other security groups	ewb-sg-bastion and ewb-sg-app-b; full IDs not visible in supplied screenshots
Key pair	ewb-lab; private key kept on administrator's computer

Identifiers are transcribed from the supplied console screenshots and setup notes. Full security-group IDs are not invented where they were not visible.

6. Route Table Configuration

A route table is a set of instructions that tells AWS where to send packets. Each VPC has a local route for its own CIDR. The peering route points traffic for the remote VPC CIDR to the peering connection. The public route table also contains a default route to the Internet Gateway.

Route table	Destination	Target	Evidence
ewb-rt-a-public	10.10.0.0/16	local	Active in screenshot
ewb-rt-a-public	0.0.0.0/0	ewb-igw-a	Active in screenshot
ewb-rt-a-public	10.20.0.0/16	pcx-0681831548570ed5c	Active in screenshot
ewb-rt-a-private	10.10.0.0/16	local	Active in screenshot
ewb-rt-a-private	10.20.0.0/16	pcx-0681831548570ed5c	Active in screenshot
ewb-rt-b-private	10.20.0.0/16	local	Configured
ewb-rt-b-private	10.10.0.0/16	pcx-0681831548570ed5c	Reported configured; detailed screenshot still required

The detailed screenshots supplied show the Active peering route in both VPC A route tables. The VPC B private route must also be checked and captured to document the return path.

7. VPC Peering Connection

The peering connection was created from VPC A to VPC B with the name **ewb-a-b-peer**. AWS initially showed Pending acceptance. The request was then accepted, and the console displayed a confirmation that the connection had been established. The peering connection ID is **pcx-0681831548570ed5c** and its current status shown in the screenshot is **Active**.

An Active status means AWS has established the peering relationship. It does not by itself prove that packets can reach the application instances. Routes and security-group rules are still required.

8. Security Group Configuration

Security group	Inbound rule	Source
ewb-sg-bastion	SSH TCP 22	Administrator public IP 178.93.83.63/32; EC2 Instance Connect prefix list also present
ewb-sg-app-a	SSH TCP 22	Bastion private IP 10.10.1.10/32
ewb-sg-app-a	ICMPv4 Echo Request	App-B private IP 10.20.1.10/32
ewb-sg-app-b	SSH TCP 22	Bastion private IP 10.10.1.10/32
ewb-sg-app-b	ICMPv4 Echo Request	App-A private IP 10.10.2.10/32

These are the intended lab rules recorded during setup. The supplied screenshots show only a partial Bastion inbound-rules view and the App-A security details panel, not every rule row for all three groups. Verify the live rules before final submission. Security groups are stateful, so response traffic for an allowed connection is automatically permitted.

9. Bastion Host and Administrative Access

A Bastion Host is a controlled entry point used to administer servers that do not have public IP addresses. In this lab, the Bastion is in VPC A's public subnet. Its private IPv4 address is 10.10.1.10 and its public IPv4 address is 13.232.237.127. App-A and App-B are private instances.

The intended path is administrator computer → Bastion → private application server. To reach App-B through the Bastion, VPC A's public route table also needs the route for 10.20.0.0/16 to the peering connection. The Bastion's SSH daemon was verified listening on TCP port 22. The browser-based EC2 Instance Connect terminal was usable.

However, the Windows Test-NetConnection screenshot shows that the laptop could not establish a TCP connection to 13.232.237.127 on port 22 (TcpTestSucceeded = False). This is a connection-path failure before SSH key authentication. A previous Bastion-to-App-A SSH attempt reached the SSH service but returned a public-key authentication error. The private key should remain on the administrator's computer and must not be copied to the Bastion.

App-A's Security details screenshot shows no IAM role attached. Therefore, Systems Manager Session Manager access is not established by the current configuration. It would require an appropriate instance role and network connectivity to Systems Manager endpoints.

10. EC2 Instance Status

The EC2 instances list screenshot shows three instances: ewb-bastion, ewb-app-a and ewb-app-b. All three are Running and each shows 3/3 status checks passed. The Bastion summary screenshot shows its public and private IP addresses. The EC2 list shows no public IPv4 DNS values for App-A and App-B.

11. Connectivity Test Results

Validation	Evidence / observation	Status
Peering connection	AWS console shows pcx-0681831548570ed5c as Active	Verified
VPC A public route	10.20.0.0/16 → peering; Active	Verified
VPC A private route	10.20.0.0/16 → peering; Active	Verified
VPC B private return route	Reported configured; detailed route screenshot not supplied	Capture proof
EC2 health	All three Running; 3/3 checks passed	Verified
Bastion sshd	Terminal shows sshd listening on port 22	Verified
Laptop → Bastion TCP 22	Test-NetConnection failed	Unresolved
App-A → App-B ping	No successful output supplied	Pending
App-B → App-A ping	No successful output supplied	Pending
SSH to App-B through Bastion	No successful login output supplied	Pending
Controlled ICMP failure/recovery	No before/after output supplied	Pending

Do not mark the pending tests as successful until actual terminal output has been captured. A ping test should show replies and packet statistics; an SSH test should show a successful login prompt.

12. Troubleshooting Summary

Laptop-to-Bastion SSH timed out: The Bastion's SSH daemon was running and listening, but the laptop's TCP test to port 22 failed. The browser terminal remained available. The issue is therefore not proven to be an SSH daemon failure; it is an unresolved network path or filtering issue between the laptop and the public EC2 address.

SSH from Bastion to App-A returned Permission denied: The request reached the SSH service, but the session did not provide an accepted key. This is an authentication issue, not the same as a network timeout. Keep the private key secure and do not copy it onto the Bastion.

App-A has no IAM role: The EC2 details show a dash in the IAM role field. SSM should not be assumed available without adding the correct instance profile and private service connectivity.

13. Remaining Actions for Full Project Completion

• Open ewb-rt-b-private and capture its Routes tab. Confirm 10.10.0.0/16 points to pcx-0681831548570ed5c and is Active.
• Establish secure access to App-A and App-B using an approved method.
• From App-A, run ping 10.20.1.10 and save the real terminal output.
• From App-B, run ping 10.10.2.10 and save the real terminal output.
• Demonstrate SSH to App-B through the Bastion without placing the private key on the Bastion.
• Temporarily remove the App-B inbound ICMP Echo Request rule from 10.10.2.10/32, verify ping fails, restore the rule and verify ping succeeds.
• Add genuine test screenshots to the evidence folder and replace any illustrative connectivity graphics used as examples.

14. Cleanup and Cost Control

Complete cleanup only after all required tests and evidence have been captured. EC2 instances, EBS storage and public IPv4 resources can incur charges while they remain allocated.

1. Terminate ewb-bastion, ewb-app-a and ewb-app-b.
2. Review EBS Volumes and Snapshots; delete lab storage that is no longer required.
3. Release any Elastic IP addresses allocated for the lab, if any.
4. Delete the ewb-lab key pair after the project no longer needs it.
5. Remove the peering routes and delete the VPC Peering connection.
6. Delete the lab security groups after their network interfaces are removed.
7. Disassociate and delete the custom route tables.
8. Delete the subnets.
9. Detach and delete ewb-igw-a.
10. Delete both VPCs after dependent resources are removed.
11. Recheck EC2, EBS, VPC and public IPv4 resources for leftovers.

15. Key Learnings and Conclusion

The lab demonstrates that private communication requires a complete network path. VPC Peering provides the connection between VPCs, route tables direct packets across it, and security groups decide which traffic is allowed. Both VPCs need appropriate routes for two-way communication. VPC Peering is non-transitive and does not automatically share Internet access or an Internet Gateway.

The core resources have been provisioned, the peering connection is Active, the supplied VPC A route screenshots show Active peering routes, and all three EC2 instances are Running with passing status checks. End-to-end application connectivity is not yet proven by the supplied evidence. Completing the remaining tests and adding genuine terminal screenshots will make the project report complete.

Appendix A – AWS Console and Terminal Evidence

The following images were supplied during the implementation session. Captions explain the visible evidence. Partial screenshots do not prove settings outside the displayed area.

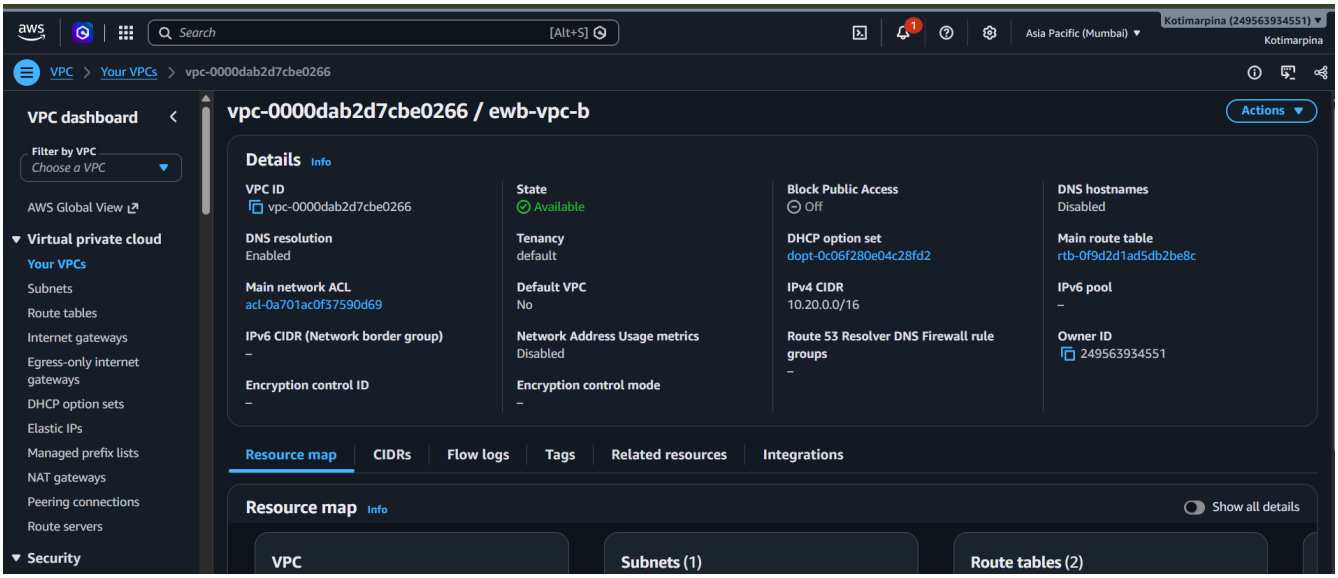


Figure A1. VPC B details, including VPC ID and IPv4 CIDR 10.20.0.0/16.

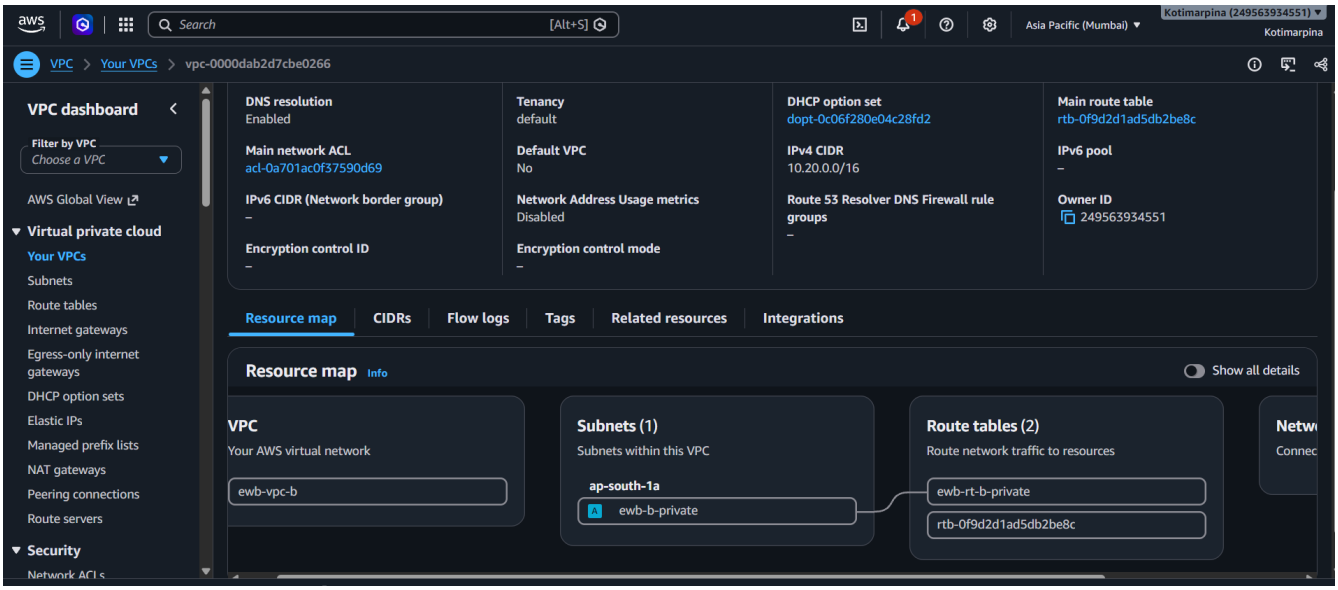


Figure A2. VPC B resource map showing the private subnet and route-table association.

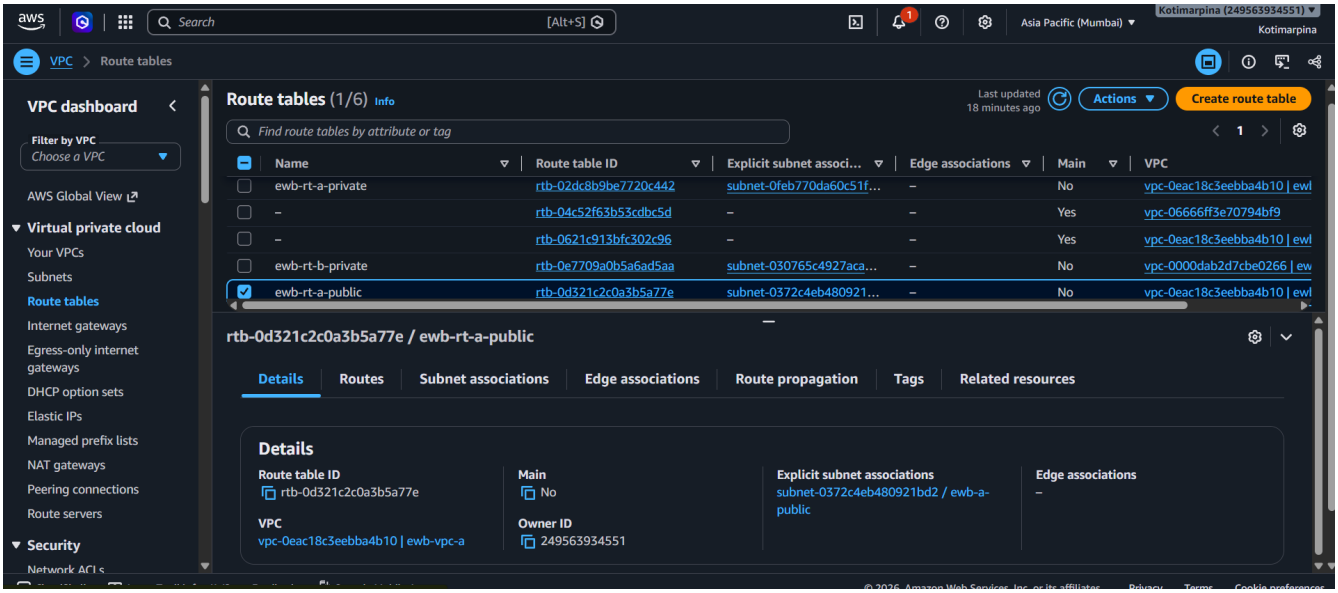


Figure A3. Route-table inventory with ewb-rt-a-public selected.

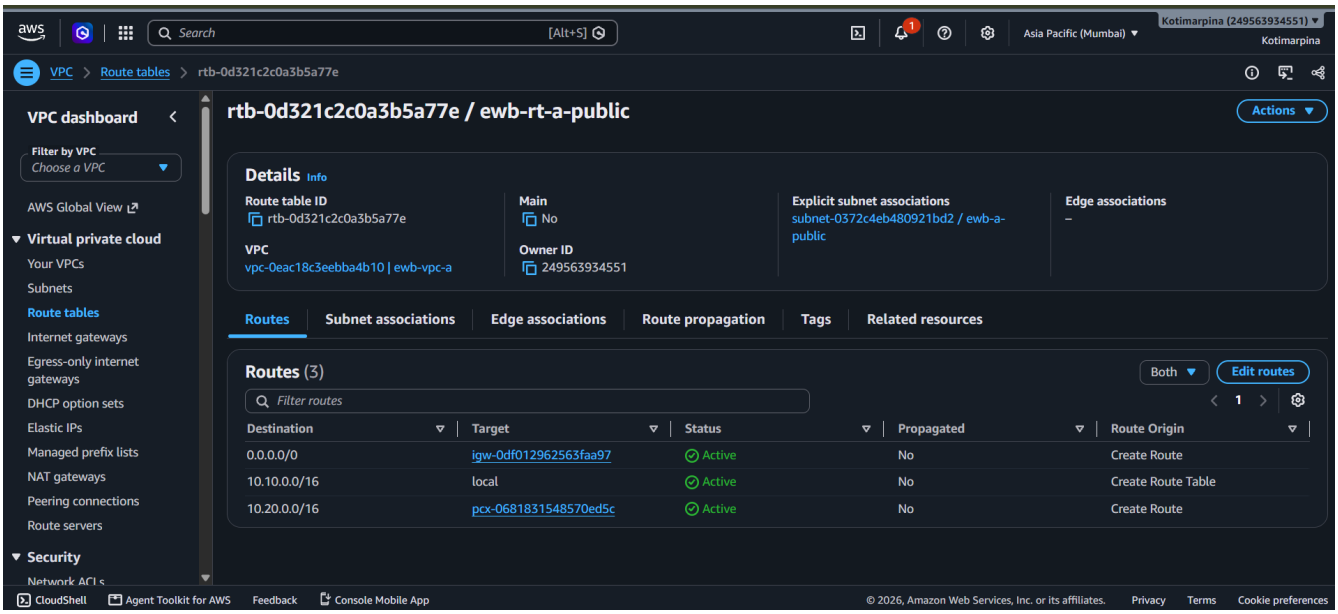


Figure A4. ewb-rt-a-public routes, including the Internet Gateway default route and Active peering route.

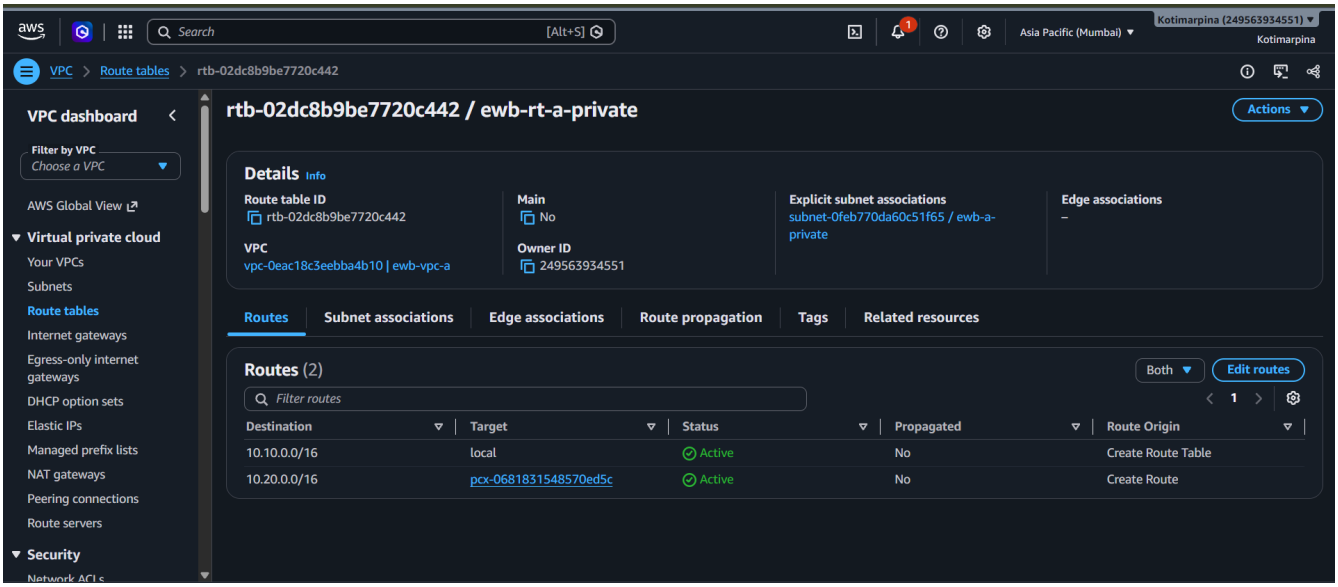


Figure A5. ewb-rt-a-private routes, including the Active route to VPC B.

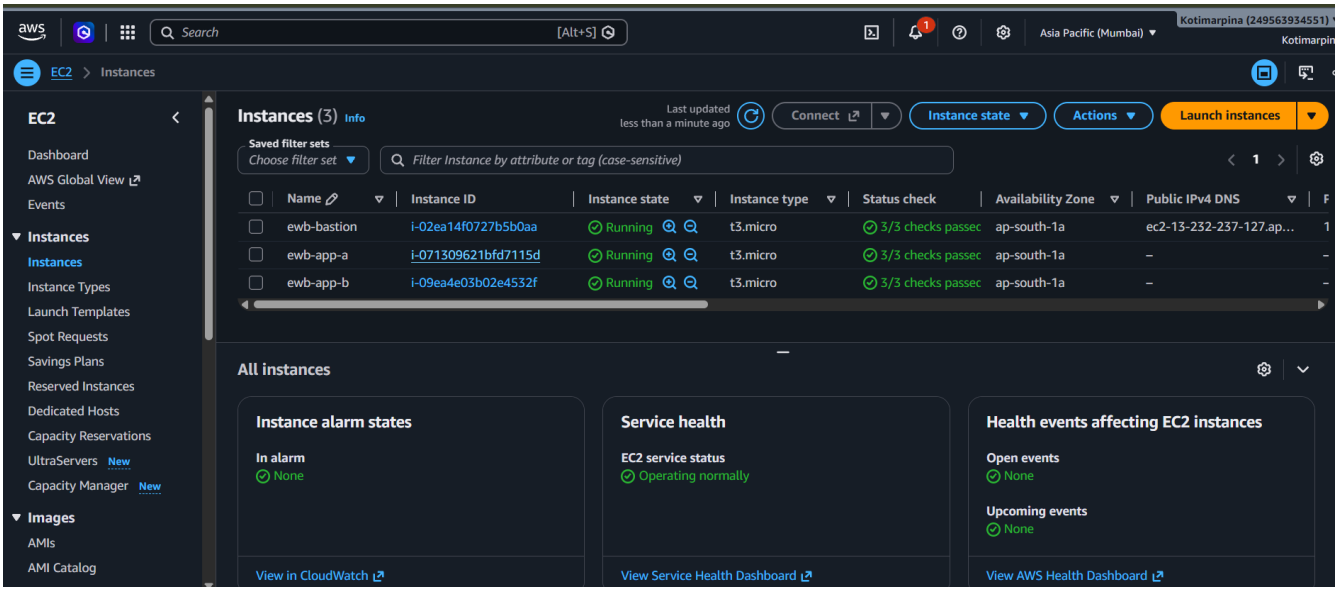


Figure A6. EC2 list showing all three instances Running with 3/3 status checks passed.

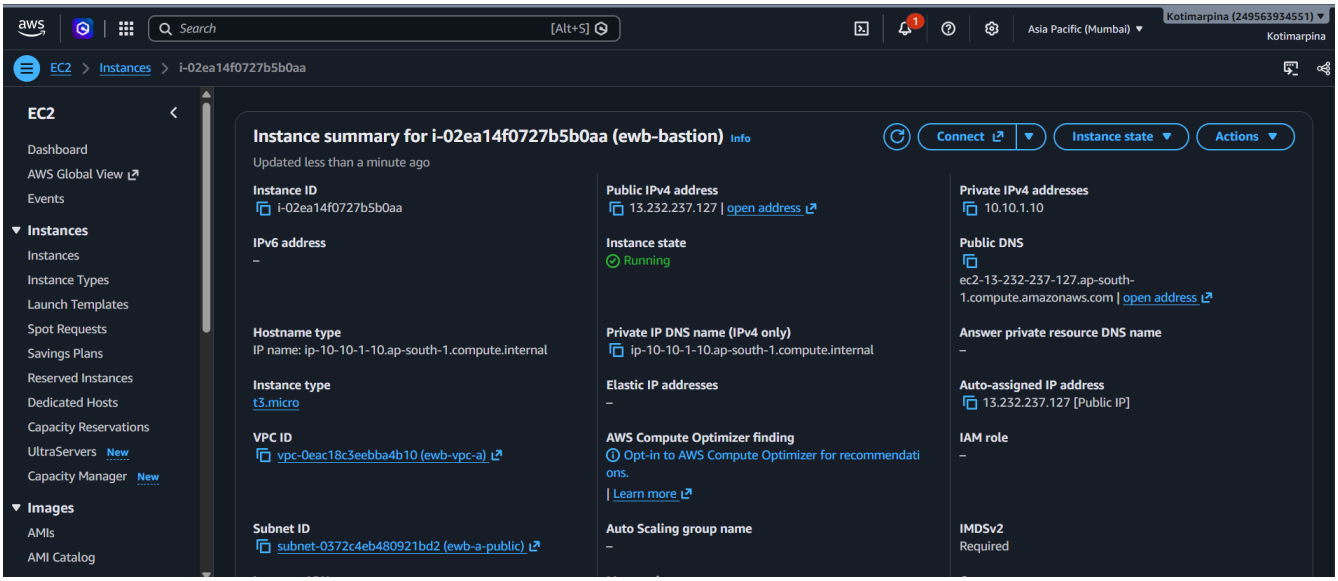


Figure A7. Bastion summary showing public IPv4 13.232.237.127 and private IPv4 10.10.1.10.

```
[ec2-user@ip-10-10-1-10 ~]$ sudo ss -lntp | grep ':22'
LISTEN 0      128          0.0.0.0:22        0.0.0.0:*    users:(("sshd",pid=1901,fd=7))
LISTEN 0      128             [::]:22           [::]:*    users:(("sshd",pid=1901,fd=8))
[ec2-user@ip-10-10-1-10 ~]$
```

Figure A8. Bastion terminal output showing sshd listening on TCP port 22.

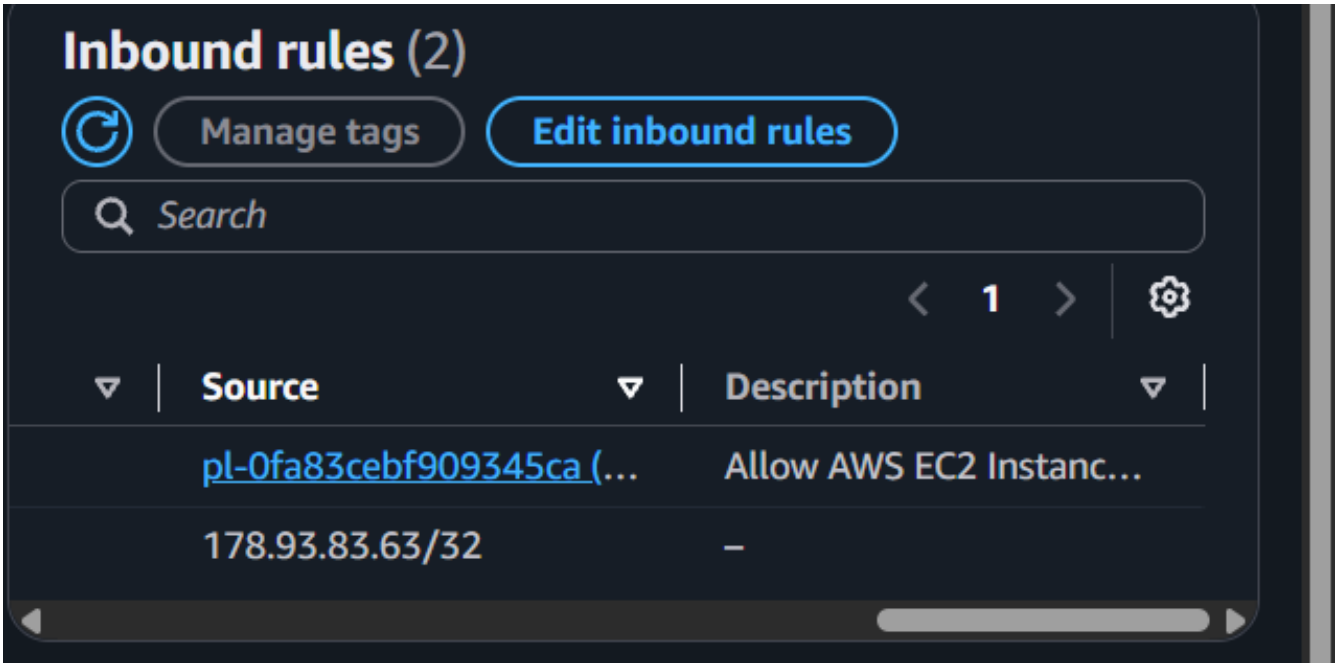


Figure A9. Partial Bastion security-group inbound rules view.

```
PS C:\Windows> Test-NetConnection 13.232.237.127 -Port 22 -InformationLevel Detailed
WARNING: TCP connect to (13.232.237.127 : 22) failed
WARNING: Ping to 13.232.237.127 failed with status: TimedOut


ComputerName            : 13.232.237.127
RemoteAddress           : 13.232.237.127
RemotePort              : 22
NameResolutionResults   : 13.232.237.127
                          ec2-13-232-237-127.ap-south-1.compute.amazonaws.com
MatchingIPsecRules      :
NetworkIsolationContext : Internet
IsAdmin                 : False
InterfaceAlias          : Wi-Fi
SourceAddress           : 10.206.126.233
NetRoute (NextHop)      : 10.206.126.88
PingSucceeded           : False
PingReplyDetails (RTT)  : 0 ms
TcpTestSucceeded        : False
```

Figure A10. Windows Test-NetConnection output showing TCP connection to Bastion port 22 failed.

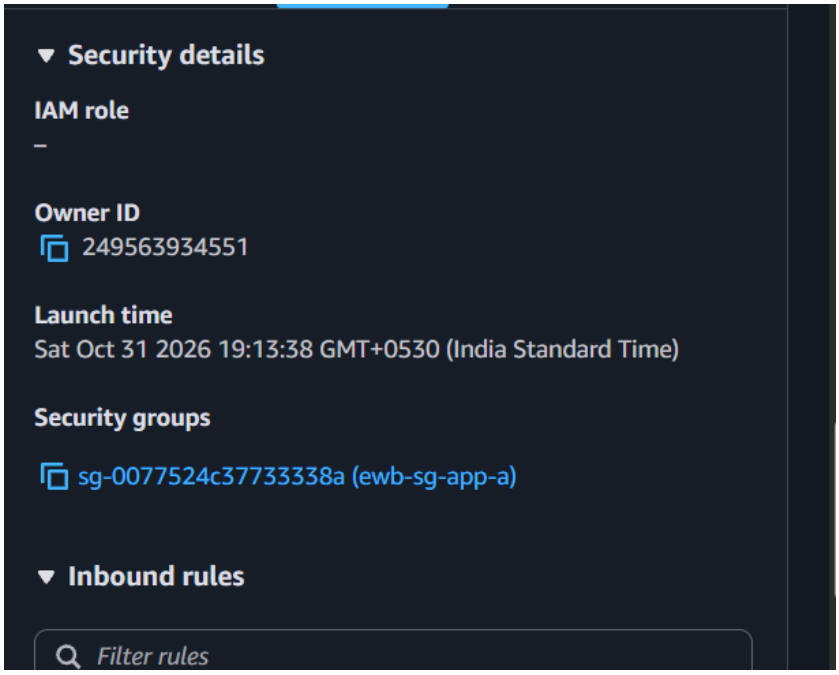


Figure A11. App-A security details showing no IAM role attached.



Figure A12. EC2 navigation panel showing the visible Network & Security menu entries.